TRAFFIC FLOW PATTERN ANALYSIS USING K-MEANS AND DBSCAN CLUSTERING

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
In
A.Y.: 2026-27

By

HAADI HASIL	**2520030297**
SATISH	**2520030302**
VEMULA GAUTAM	**2520030316**

Under the Esteemed guidance of
Mr. Gaddam Ravindra Babu
Assistant Professor, CS&IT

Department of Computer Science

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Abstract

Road traffic congestion is a major problem for growing cities. It increases travel time, fuel use and emissions, and delays emergency services. Road sensors continuously record average speed, traffic volume and lane occupancy, but these readings do not come with labels that say whether a road is flowing freely or is jammed, and some readings are corrupted or abnormal. This project proposes a Traffic Flow Pattern Analysis system that uses unsupervised machine learning to discover traffic states directly from sensor data without relying on labelled data.

The system uses three traffic parameters: speed (km/h), traffic volume (vehicles per interval) and occupancy (fraction of time a detector is covered). The data is pre-processed through numeric conversion, removal of missing, non-numeric and physically impossible values, and feature standardization. Two clustering algorithms, K-Means and DBSCAN (Density-Based Spatial Clustering of Applications with Noise), are implemented and compared. K-Means partitions observations around K centroids, while DBSCAN groups observations that lie in dense regions and labels isolated observations as noise.

The complete workflow, covering data cleaning, exploratory data analysis, clustering, PCA visualization, evaluation and downloadable results, is delivered as an interactive Streamlit web application. On the project dataset, both algorithms recovered free-flow, moderate and congested traffic states, while DBSCAN additionally isolated the two incident-like observations as outliers. The project demonstrates how unsupervised learning can support traffic monitoring and data-driven decisions in smart-city systems.

1. Introduction

1.1 Purpose of the Project

Traffic congestion affects travel time, fuel consumption, air quality and road safety. Modern roads are equipped with inductive loops, radar units and cameras that report speed, vehicle count and occupancy every few minutes. Classical traffic-flow theory explains that as more vehicles use a road, volume and occupancy rise and, beyond a critical point, average speed falls and the road moves from free flow into congestion. Manually reviewing these continuously collected readings to find congestion and unusual events is slow and becomes impractical as data grows.

The project is motivated by the need for an automated, data-driven way to identify traffic states and unusual readings. The main stakeholders include city traffic-management cells, transport planners, smart-city teams and researchers. Because sensor data rarely carries labels such as "congested", an unsupervised machine-learning approach is appropriate: it groups similar observations together, and the analyst then interprets each group.

1.2 Problem Statement

The project is formulated as an unsupervised machine-learning clustering problem. Given traffic observations containing speed, traffic volume and occupancy, the system must group the observations into meaningful traffic states and identify observations that do not belong to any normal state, without depending on pre-labelled classes. The expected output is a cluster label for each valid observation (interpreted as free-flow, moderate or congested traffic) and, for DBSCAN, a noise label (-1) marking possible incidents or sensor faults.

1.3 Existing Methods and Disadvantages

Traffic states are commonly identified using fixed threshold rules, for example "speed below 30 km/h means congestion". Such thresholds do not adapt to different roads, speed limits or times of day, and they examine one variable at a time. Manual inspection of sensor feeds is also slow and inconsistent. Supervised classifiers need labelled examples of each traffic state, which are usually unavailable for sensor data.

A further difficulty is that real sensor data contains faulty records (blank or text values) and genuine anomalies. Some anomalies are not extreme in any single variable but are unusual only in their combination, for example very low speed with very few vehicles. Single-variable rules miss such cases. The project therefore considers unsupervised clustering on all features together. K-Means is simple and fast but must be told the number of clusters and assigns every observation to a cluster. DBSCAN finds the number of clusters automatically and can label sparse observations as noise. Using both allows their behaviour to be compared; the project does not assume one method is always superior.

1.4 Proposed System

The proposed system implements a complete machine-learning pipeline for traffic-flow pattern analysis. Traffic readings containing speed, volume and occupancy are loaded from a default CSV file or a user-uploaded file. Invalid values are converted, detected and removed, and the remaining observations are explored using descriptive statistics, a correlation matrix, histograms, box plots and scatter plots. The features are standardized and clustered with K-Means and DBSCAN. Results are projected into two dimensions using Principal Component Analysis (PCA), profiled per cluster, evaluated using Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Score, and made available for download. All steps run inside an interactive Streamlit web application in which K, eps and min_samples can be adjusted with sliders.

1.5 Research Objectives (at least 4)

1. Prepare and validate the project traffic dataset containing 24 five-minute observations and three numerical traffic features.
2. Detect and remove missing, non-numeric and physically impossible values, and standardize the numerical features before clustering.
3. Implement K-Means with K = 3, random_state = 42 and n_init = 10 to discover traffic states.
4. Implement DBSCAN with eps = 0.65 and min_samples = 4 for density-based clustering and outlier detection.
5. Evaluate both models using Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Score, and compare the clusters with the reference illustrative_state field.
6. Deploy the workflow as an interactive Streamlit application with downloadable results, and document the practical limitations of the current implementation.

Note: The dataset used is the sample file supplied with the project (data/traffic_sample.csv), not a Kaggle dataset. It is a small demonstration dataset. If a public dataset is required, the same application can be run on a public traffic dataset such as Metro Interstate Traffic Volume or PeMS by uploading the CSV.

2. Literature Survey (recent studies from 2-3 years)

The following ten recent and directly relevant studies were reviewed. They cover traffic congestion pattern identification, clustering of traffic data with K-Means and DBSCAN, DBSCAN parameter selection, and anomaly detection in transport sensor data.

1. Traffic congestion patterns in the urban road network: (Dammam metropolitan area)

Reference with DOI: Almatar, K. M. (2023), Ain Shams Engineering Journal, 14(3), 101886. DOI: 10.1016/j.asej.2022.101886

Problem or gap addressed: Traffic congestion hinders sustainable urban transport, and the most congested parts of a network and their link to driver demand must be identified.

Objective of the Research: Identify the most congested areas of the urban road network and examine how they relate to driver demand.

Focus of the paper (Methods used): Floating Car Data used to measure congestion; congestion clusters assessed across the network and tested statistically for correlation.

Inference: Congestion was more severe and widespread in evening peak hours than in the morning, and the clustering approach effectively identified congestion patterns.

Recommendations for future research: The author suggests using the results as the basis for a framework that describes and predicts future congestion problems.

2. Identifying Traffic Congestion Patterns of Urban Road Network Based on Traffic Performance Index

Reference with DOI: Zang, J., Jiao, P., Liu, S., Zhang, X., Song, G., & Yu, L. (2023), Sustainability, 15(2), 948. DOI: 10.3390/su15020948

Problem or gap addressed: Most studies examine traffic in limited districts rather than long-term congestion across a whole city road network.

Objective of the Research: Identify time-varying congestion patterns of Beijing's road network using the Traffic Performance Index (TPI) over 699 days.

Focus of the paper (Methods used): Self-Organizing Map (SOM) clustering with an automatic cluster-number algorithm applied to daily TPI profiles.

Inference: Eight congestion patterns were found (e.g. Mondays, Fridays, ordinary and congested weekdays, weekends, festivals); peak-hour congestion in 2021 rose 7.15% over 2019.

Recommendations for future research: Suggested direction: combine network-level pattern clustering with segment-level speed, volume and occupancy data.

3. A Clustering-Based Approach to Detecting Critical Traffic Road Segments in Urban Areas

Reference with DOI: Košanin, I., Gnjatović, M., Maček, N., & Joksimović, D. (2023), Axioms, 12(6), 509. DOI: 10.3390/axioms12060509

Problem or gap addressed: Detecting road segments with prolonged, high accident risk; generic clustering evaluation criteria do not reflect the traffic domain.

Objective of the Research: Propose a parameter-free clustering approach and a domain-specific criterion for evaluating clustering results.

Focus of the paper (Methods used): Distance-threshold clustering with bottom-up merging; threshold chosen automatically by knee-point detection (adapted Kneedle algorithm).

Inference: Applied to three years of accident data from three Serbian cities; clusters were stable across periods (0.990 in Belgrade) and agreed with traffic-camera locations.

Recommendations for future research: Suggested direction: knee-point detection can be used to select DBSCAN's eps from a k-distance plot.

4. STRP-DBSCAN: A Parallel DBSCAN Algorithm Based on Spatial-Temporal Random Partitioning for Clustering Trajectory Data

Reference with DOI: An, X., Wang, Z., Wang, D., Liu, S., Jin, C., Xu, X., & Cao, J. (2023), Applied Sciences, 13(20), 11122. DOI: 10.3390/app132011122

Problem or gap addressed: DBSCAN is slow on large trajectory data and highly sensitive to its Eps and MinPts parameters.

Objective of the Research: Speed up parallel DBSCAN for trajectory data and automate the choice of its parameters.

Focus of the paper (Methods used): Spatial-temporal random partitioning for parallel DBSCAN; deep reinforcement learning (PER-SAC) for automatic parameter tuning.

Inference: Clustering time was reduced by up to 96.2% versus parallel DBSCAN, and automatic tuning improved accuracy by up to 8.8%.

Recommendations for future research: Suggested direction: automatic eps/min_samples tuning is needed when DBSCAN is applied to larger traffic datasets.

5. A Data-Mining Interpretation Method of Pavement Dynamic Response Signal by Combining DBSCAN and Findpeaks Function

Reference with DOI: Liu, H., Yao, R., Cui, C., & Zhao, J. (2024), Sensors, 24(3), 939. DOI: 10.3390/s24030939

Problem or gap addressed: Strain sensor data from pavements under heavy traffic fluctuates and contains abnormal values.

Objective of the Research: Develop a method that detects anomalies and extracts strain peaks from in-pavement sensor signals.

Focus of the paper (Methods used): Signal denoising, DBSCAN improved with the K-Dist method for anomaly detection, and a refined findpeaks function for feature extraction.

Inference: The enhanced DBSCAN effectively detected abnormal sensor data, showing DBSCAN's value for cleaning traffic-related sensor streams.

Recommendations for future research: Suggested direction: use density-based anomaly detection as a real-time data-quality step in traffic monitoring systems.

6. Proposal of a Machine Learning Approach for Traffic Flow Prediction

Reference with DOI: Berlotti, M., Di Grande, S., & Cavalieri, S. (2024), Sensors, 24(7), 2348. DOI: 10.3390/s24072348

Problem or gap addressed: Congestion is increasing with urbanization, and many urban areas lack traffic sensors for forecasting.

Objective of the Research: Predict traffic flow, including in urban areas without sensors.

Focus of the paper (Methods used): Two-level approach: unsupervised clustering extracts patterns from sensor data, followed by supervised machine-learning prediction models.

Inference: Clustering-derived traffic patterns are a useful first stage for prediction and allow flow estimation where no sensors exist.

Recommendations for future research: Suggested direction: use discovered traffic clusters as inputs to forecasting models.

7. Clustering Algorithms to Analyse Smart City Traffic Data

Reference with DOI: Kumari, P. M. K., Manjaiah, D. H., & Ashwini, K. M. (2024), International Journal of Advanced Computer Science and Applications, 15(8). DOI: 10.14569/IJACSA.2024.0150811

Problem or gap addressed: Extracting meaningful traffic patterns from large historical datasets is difficult for smart-city traffic management.

Objective of the Research: Analyse temporal patterns in urban traffic and identify factors affecting traffic flow.

Focus of the paper (Methods used): DBSCAN, K-Means, Affinity Propagation, Mean Shift and Gaussian Mixture applied to hourly traffic counts from Aarhus; evaluated with Silhouette Score and Dunn Index.

Inference: DBSCAN gave the best cluster quality (Silhouette 0.3664), followed by K-Means (0.3119); DBSCAN, K-Means and GMM trained fastest.

Recommendations for future research: Use big-data technologies for GPS and detector data, and run comparative clustering studies across multiple cities.

8. Transformer model-based multi-scale fine-grained identification and classification of regional traffic states

Reference with DOI: Zhang, J., & Hu, G. (2024), PeerJ Computer Science, 10, e2625. DOI: 10.7717/peerj-cs.2625

Problem or gap addressed: Conventional traffic-state estimation uses only four to six classes, which misses subtle changes after congestion-reduction measures.

Objective of the Research: Classify regional traffic states into sixteen fine-grained levels.

Focus of the paper (Methods used): Transformer encoder for feature extraction, K-Means clustering into 16 clusters and non-dominated sorting to rank them, using SUMO simulation data.

Inference: Transformer + K-Means outperformed PCA + K-Means and Fuzzy C-Means on Silhouette, Davies-Bouldin and Calinski-Harabasz indices.

Recommendations for future research: Validate on real-world data, integrate other deep-learning architectures and develop real-time traffic management systems.

9. Empirical analysis of traffic patterns: Leveraging machine learning techniques for in-depth insights

Reference with DOI: Lahijanian, Z., Isaenko, N., Fusco, G., & Colombaroni, C. (2025), Transportation Research Procedia (AIIT 4th International Conference). ScienceDirect PII: S2352146525005757

Problem or gap addressed: Daily calibration of traffic models is impractical as datasets grow; day types with distinct traffic patterns are needed.

Objective of the Research: Group days into types with distinct traffic patterns without pre-selecting the groups.

Focus of the paper (Methods used): Eleven months of motorway traffic time series cleaned and clustered with K-Means, DBSCAN and Affinity Propagation.

Inference: K-Means and DBSCAN performed comparably well and identified six day types, separating school/non-school periods and weekdays/weekends.

Recommendations for future research: Apply the method daily to detect demand and supply anomalies for mobility-agency planning.

10. Clustering Based Approach for Enhanced Characterization of Anomalies in Traffic Flows

Reference with DOI: Khasawneh, M., & Awasthi, A. (2026), Future Transportation, 6(1), 11. DOI: 10.3390/futuretransp6010011

Problem or gap addressed: Unusually high or low traffic volumes signal incidents or congestion and need to be identified and characterized.

Objective of the Research: Develop a clustering-based method to characterize traffic-flow anomalies in space and time.

Focus of the paper (Methods used): Elliptic Envelope, Isolation Forest and Local Outlier Factor for anomaly detection; K-Means and agglomerative clustering of anomalies over 213 city zones in Montréal.

Inference: LOF performed consistently well; K-Means generally outperformed agglomerative clustering (average Silhouette 0.73) and zoning cut runtime by 42%.

Recommendations for future research: Add optimization (GA, PSO) for signal timing and benchmark DBSCAN, GMM, spectral clustering and HDBSCAN under identical settings.

Summary of the literature survey

S.No.	Title	Reference with DOI	Focus (Methods used)	Inference
1	Traffic congestion patterns in the urban road network: (Dammam metropolitan area)	Almatar, K. M. (2023), Ain Shams Engineering Journal, 14(3), 101886. DOI: 10.1016/j.asej.2022.101886	Floating Car Data used to measure congestion; congestion clusters assessed across the network and tested statistically for correlation.	Congestion was more severe and widespread in evening peak hours than in the morning, and the clustering approach effectively identified congestion patterns.
2	Identifying Traffic Congestion Patterns of Urban Road Network Based on Traffic Performance Index	Zang, J., Jiao, P., Liu, S., Zhang, X., Song, G., & Yu, L. (2023), Sustainability, 15(2), 948. DOI: 10.3390/su15020948	Self-Organizing Map (SOM) clustering with an automatic cluster-number algorithm applied to daily TPI profiles.	Eight congestion patterns were found (e.g. Mondays, Fridays, ordinary and congested weekdays, weekends, festivals); peak-hour congestion in 2021 rose 7.15% over 2019.
3	A Clustering-Based Approach to Detecting Critical Traffic Road Segments in Urban Areas	Košanin, I., Gnjatović, M., Maček, N., & Joksimović, D. (2023), Axioms, 12(6), 509. DOI: 10.3390/axioms12060509	Distance-threshold clustering with bottom-up merging; threshold chosen automatically by knee-point detection (adapted Kneedle algorithm).	Applied to three years of accident data from three Serbian cities; clusters were stable across periods (0.990 in Belgrade) and agreed with traffic-camera locations.
4	STRP-DBSCAN: A Parallel DBSCAN Algorithm Based on Spatial-Temporal Random Partitioning for Clustering Trajectory Data	An, X., Wang, Z., Wang, D., Liu, S., Jin, C., Xu, X., & Cao, J. (2023), Applied Sciences, 13(20), 11122. DOI: 10.3390/app132011122	Spatial-temporal random partitioning for parallel DBSCAN; deep reinforcement learning (PER-SAC) for automatic parameter tuning.	Clustering time was reduced by up to 96.2% versus parallel DBSCAN, and automatic tuning improved accuracy by up to 8.8%.
5	A Data-Mining Interpretation Method of Pavement Dynamic Response Signal by Combining DBSCAN and Findpeaks Function	Liu, H., Yao, R., Cui, C., & Zhao, J. (2024), Sensors, 24(3), 939. DOI: 10.3390/s24030939	Signal denoising, DBSCAN improved with the K-Dist method for anomaly detection, and a refined findpeaks function for feature extraction.	The enhanced DBSCAN effectively detected abnormal sensor data, showing DBSCAN's value for cleaning traffic-related sensor streams.
6	Proposal of a Machine Learning Approach for Traffic Flow Prediction	Berlotti, M., Di Grande, S., & Cavalieri, S. (2024), Sensors, 24(7), 2348. DOI: 10.3390/s24072348	Two-level approach: unsupervised clustering extracts patterns from sensor data, followed by supervised machine-learning prediction models.	Clustering-derived traffic patterns are a useful first stage for prediction and allow flow estimation where no sensors exist.
7	Clustering Algorithms to Analyse Smart City Traffic Data	Kumari, P. M. K., Manjaiah, D. H., & Ashwini, K. M. (2024), International Journal of Advanced Computer Science and Applications, 15(8). DOI: 10.14569/IJACSA.2024.0150811	DBSCAN, K-Means, Affinity Propagation, Mean Shift and Gaussian Mixture applied to hourly traffic counts from Aarhus; evaluated with Silhouette Score and Dunn Index.	DBSCAN gave the best cluster quality (Silhouette 0.3664), followed by K-Means (0.3119); DBSCAN, K-Means and GMM trained fastest.
8	Transformer model-based multi-scale fine-grained identification and classification of regional traffic states	Zhang, J., & Hu, G. (2024), PeerJ Computer Science, 10, e2625. DOI: 10.7717/peerj-cs.2625	Transformer encoder for feature extraction, K-Means clustering into 16 clusters and non-dominated sorting to rank them, using SUMO simulation data.	Transformer + K-Means outperformed PCA + K-Means and Fuzzy C-Means on Silhouette, Davies-Bouldin and Calinski-Harabasz indices.

9	Empirical analysis of traffic patterns: Leveraging machine learning techniques for in-depth insights	Lahijanian, Z., Isaenko, N., Fusco, G., & Colombaroni, C. (2025), Transportation Research Procedia (AIIT 4th International Conference). ScienceDirect PII: S2352146525005757	Eleven months of motorway traffic time series cleaned and clustered with K-Means, DBSCAN and Affinity Propagation.	K-Means and DBSCAN performed comparably well and identified six day types, separating school/non-school periods and weekdays/ weekends.
10	Clustering Based Approach for Enhanced Characterization of Anomalies in Traffic Flows	Khasawneh, M., & Awasthi, A. (2026), Future Transportation, 6(1), 11. DOI: 10.3390/ futuretransp6010011	Elliptic Envelope, Isolation Forest and Local Outlier Factor for anomaly detection; K-Means and agglomerative clustering of anomalies over 213 city zones in Montréal.	LOF performed consistently well; K-Means generally outperformed agglomerative clustering (average Silhouette 0.73) and zoning cut runtime by 42%.

Table 2.1: Literature survey summary

Research gap: Most studies apply clustering to large city-wide datasets or focus on either congestion patterns or anomalies separately. Few provide a simple, interactive tool that cleans raw sensor readings, discovers traffic states and flags incident-like observations in one workflow, while comparing centroid-based and density-based clustering on the same data. This project addresses that gap at a demonstration scale.

3. Methodology

3.1 Proposed System Flow Diagram

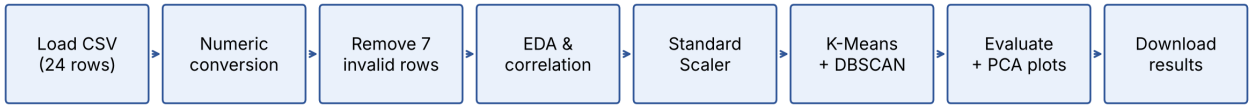


Figure 3.1: Proposed system flow

The implemented system follows the sequence: dataset loading → numeric conversion → removal of invalid rows → exploratory data analysis → standardization → K-Means and DBSCAN clustering → PCA visualization and evaluation → cluster profiling → CSV download through the Streamlit interface.

No categorical encoding, train/test split or automated hyperparameter search is implemented in app.py. Clustering is unsupervised, so the full set of valid observations is used for fitting and for computing the internal evaluation metrics.

3.2 Dataset Description

The dataset used for this project is traffic_sample.csv, supplied in the project's data folder. It contains 24 five-minute observations recorded on 2026-10-06 from 09:00 to 10:55 across three road segments (Road-A, Road-B and Road-C), with six columns: timestamp, three numerical traffic variables (speed_kmh, traffic_volume, occupancy), road_segment and illustrative_state. The three numerical variables are used as machine-learning features. The illustrative_state field is a reference label used only to check results: 5 Free-flow, 5 Moderate, 5 Congested, 2 Incident-like and 7 rows marked as invalid (blank or text values such as "abc", "unknown" and "high").

Column	Type	Description
timestamp	Date-time	Time of the reading (5-minute interval)
speed_kmh	Numeric	Average vehicle speed in km/h
traffic_volume	Numeric	Vehicles passing the sensor in the interval
occupancy	Numeric (0–1)	Fraction of time the detector is occupied
road_segment	Categorical	Road-A, Road-B or Road-C
illustrative_state	Categorical	Reference label for checking results (not used for training)

Table 3.1: Dataset attributes

3.3 Experimental Setup

The project is implemented in Python 3 as a Streamlit web application (app.py). It uses Pandas for CSV loading and cleaning, NumPy for numerical operations, Scikit-learn for StandardScaler, KMeans, DBSCAN, PCA and the evaluation metrics, and Plotly Express for interactive charts. The minimum package versions listed in requirements.txt are streamlit ≥ 1.35, pandas ≥ 2.0, numpy ≥ 1.24, scikit-learn ≥ 1.3 and plotly ≥ 5.18. The application is started with "streamlit run app.py" and opened at http://localhost:8501. The static figures in this report were regenerated with Matplotlib using the same parameters. Hardware specifications are not stated in the project files, so they are not inferred here.

3.4 Data Preprocessing

- **Data inspection and schema validation:** the three feature columns are checked and previewed.
- **Type conversion:** pd.to_numeric(errors="coerce") converts text such as "abc" to missing values.
- **Missing-value handling:** rows with a missing feature value are removed (not imputed); removed rows are kept for download as invalid_rows_removed.csv.
- **Range validation:** rows with negative values or occupancy greater than 1 are flagged as impossible.
- **Duplicate removal:** not performed in the current code.
- **Outlier treatment:** outliers are not removed; DBSCAN identifies them as noise during clustering.
- **Categorical encoding:** not required, because all selected features are numerical.
- **Scaling:** StandardScaler transforms each feature to zero mean and unit variance.
- **Train/test split:** not applicable to this unsupervised clustering workflow.

3.5 Exploratory Data Analysis

The application's EDA tab shows a dataset preview, descriptive statistics, a missing-value table, a correlation matrix, histograms, box plots and a scatter plot coloured by road segment. The key plots are reproduced below.

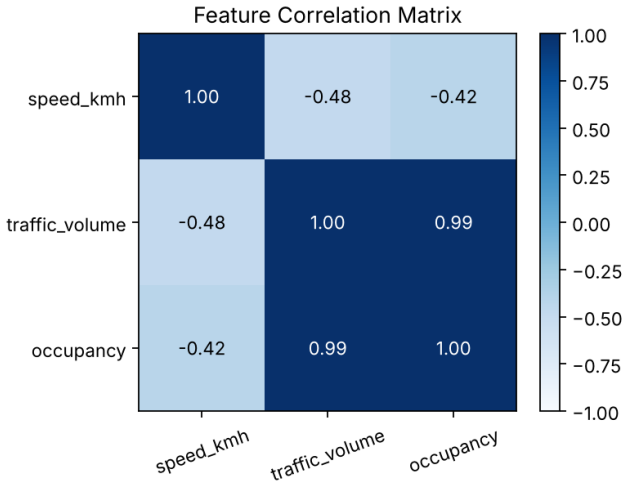


Figure 3.2: Correlation matrix of traffic features

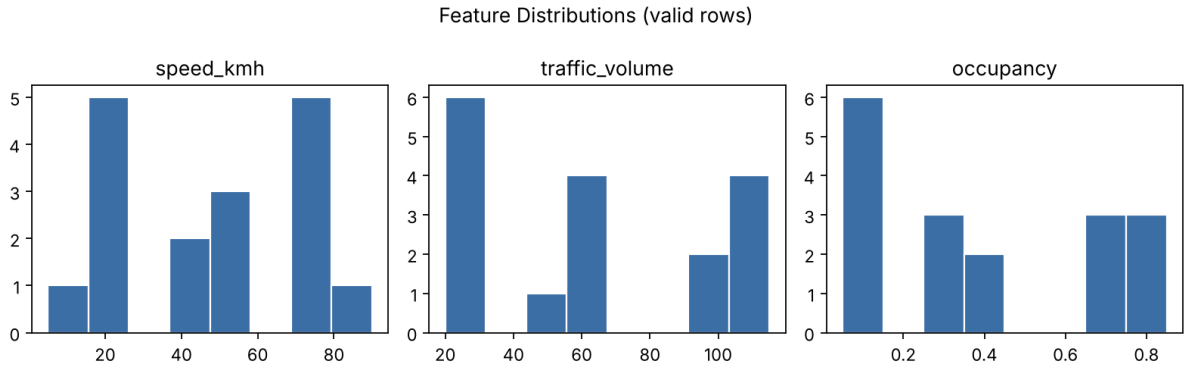


Figure 3.3: Feature distributions (valid rows)

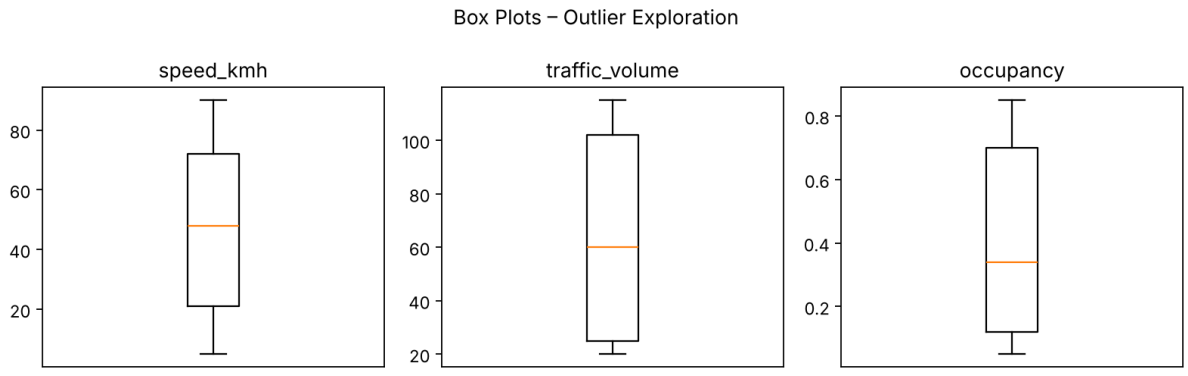


Figure 3.4: Box plots for outlier exploration

Traffic volume and occupancy are almost perfectly correlated (r = 0.99), while speed is negatively correlated with volume (−0.48) and occupancy (−0.42). The histograms show separate groups of values rather than a single peak, suggesting distinct traffic regimes. The box plots show no points beyond the whiskers, which means the abnormal observations are not extreme in any single feature; they are unusual only in their combination of values, so a multivariate method such as DBSCAN is needed to detect them.

3.6 Feature Engineering and Feature Selection (if applicable)

No new derived features are created. The selected feature set consists of speed_kmh, traffic_volume and occupancy; the sidebar lets the user choose any two or three of these. timestamp and road_segment are kept in the result table for reporting but are not passed to the models, and illustrative_state is kept only as a reference. All selected features are standardized with StandardScaler.

3.7 Model Building and Training

Two unsupervised clustering models are implemented. K-Means uses n_clusters = K (default 3), random_state = 42 and n_init = 10. It repeatedly assigns each observation to the nearest centroid and updates the centroids to minimize the within-cluster sum of squared distances. DBSCAN uses eps = 0.65 and min_samples = 4 by default. A point with at least min_samples neighbours within distance eps is a core point; clusters grow from connected core points, and points not reachable from any core point are labelled −1 (noise). PCA with two components is used only to visualize the clusters. These models do not use a loss function with gradient-based training; they are fitted on all valid standardized observations.

3.8 Hyperparameter Tuning (if applicable)

No grid search, random search or cross-validation is implemented in app.py. Instead, sidebar sliders allow manual tuning of K (2–8), eps (0.10–2.00) and min_samples (2–15). For this report, an additional sweep of K from 2 to 6 was run outside the application to check the choice of K; its results are shown in Section 4.2. The defaults K = 3, eps = 0.65 and min_samples = 4 are therefore manually chosen values, not the output of systematic optimization.

3.9 Model Evaluation

Because no ground-truth labels are used for training, the application evaluates cluster quality with internal metrics: Silhouette Score (higher is better), Davies-Bouldin Index (lower is better) and Calinski-Harabasz Score (higher is better). For DBSCAN, noise points are excluded before computing these metrics. For this report, the clusters were additionally compared with the reference illustrative_state field using the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI), and the detection of the two incident-like rows was scored with accuracy, precision, recall and F1-score. These external checks are not part of app.py.

4. Results and Discussion

4.1 Preprocessing Results

The dataset contains 24 rows and 6 columns. Each of the three features has 3 unusable entries (blank or text): speed_kmh (3), traffic_volume (3) and occupancy (3), affecting 7 rows in total. These 7 rows were removed, leaving 17 valid observations. No negative values or occupancy values above 1 were found. StandardScaler then standardized the three features. The final feature count is 3.

Feature	Count	Mean	Std	Min	Median	Max
speed_kmh	17	47.29	26.29	5.00	48.00	90.00
traffic_volume	17	64.00	36.55	20.00	60.00	115.00
occupancy	17	0.40	0.29	0.05	0.34	0.85

Table 4.1: Descriptive statistics after cleaning

4.2 Model Performance

Model	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Clusters	Noise	ARI	NMI
K-Means (K=3)	0.6559	0.5729	24.99	3	0	0.7915	0.8353
DBSCAN (eps=0.65, min=4)	0.8633*	0.1620*	459.18*	3	2	1.0000	1.0000

*Table 4.2: Clustering performance (*DBSCAN metrics computed on the 15 non-noise points)*

Model	Accuracy	Precision	Recall	F1-score	Confusion matrix
K-Means	0.8824	0.0000	0.0000	0.0000	[[15, 0], [2, 0]]
DBSCAN	1.0000	1.0000	1.0000	1.0000	[[15, 0], [0, 2]]

Table 4.3: Detection of the 2 incident-like observations (K-Means has no noise label)

K	Silhouette	Davies-Bouldin	Calinski-Harabasz	Observation
2	0.600	0.621	25.8	Merges two traffic states
3	0.656	0.573	25.0	Matches the three traffic states
4	0.708	0.360	43.8	Separates one anomaly
5	0.762	0.125	309.8	Each anomaly becomes a single-point cluster
6	0.666	0.227	403.5	Starts splitting normal states

Table 4.4: K-Means sweep over K (run for this report)

K = 5 has the best internal scores only because it places each anomaly in its own one-point cluster, which confirms that the data contains outliers K-Means cannot label as noise. K = 3 was kept because it gives the interpretable three-state result.

4.3 Visual Results and discussion

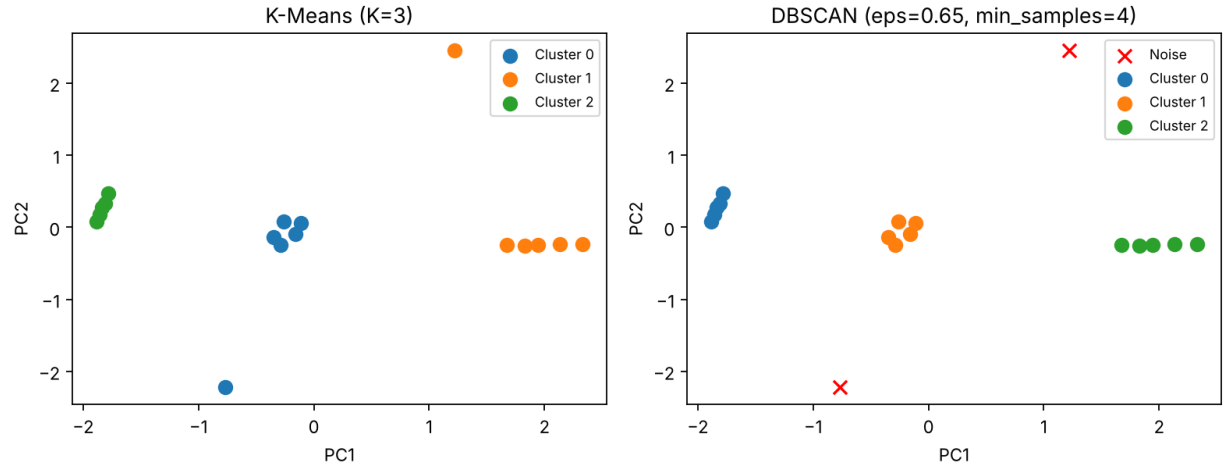


Figure 4.1: K-Means and DBSCAN clusters in PCA space (red × = DBSCAN noise)

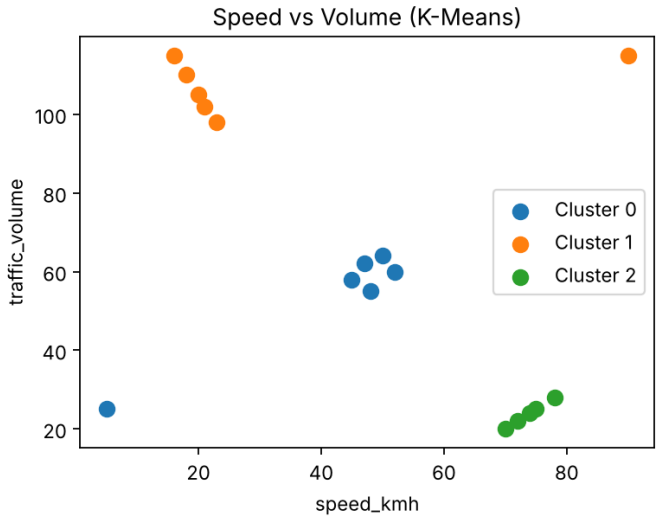


Figure 4.2: Speed versus volume coloured by K-Means cluster

Cluster	Rows	Mean speed (km/h)	Mean volume	Mean occupancy	Interpretation
K-Means 2	5	73.8	23.8	0.11	Free-flow
K-Means 0	6	41.2	54.0	0.29	Moderate + 1 stalled-vehicle anomaly
K-Means 1	6	31.3	107.5	0.75	Congested + 1 sensor-fault anomaly

Table 4.5: K-Means cluster profiles

The PCA projection explains 99.9% of the variance (PC1 76.7%, PC2 23.2%), so the two-dimensional plots represent the data faithfully. On the 17 valid observations, both algorithms placed the 15 normal readings into the same three groups: free-flow (about 74 km/h, 24 vehicles, 0.11 occupancy), moderate (about 48 km/h, 60 vehicles, 0.34 occupancy) and congested (about 20 km/h, 106 vehicles, 0.73 occupancy).

The difference lies in the two incident-like readings. K-Means placed the 5 km/h / 25-vehicle reading in the moderate cluster and the 90 km/h / 115-vehicle reading in the congested cluster, which distorted those clusters' average speeds and hid both anomalies (ARI 0.79, anomaly F1 0.00). DBSCAN labelled both readings as noise and matched the reference states exactly (ARI 1.00, anomaly F1 1.00), with confusion matrix [[15, 0], [0, 2]]. Part of DBSCAN's higher internal scores comes from excluding noise points before scoring. Because the dataset is very small and was designed with well-separated states, these figures demonstrate the method on the supplied data only and should not be read as an estimate of performance on large real-world traffic datasets.

5. Conclusion

The project demonstrates an unsupervised machine-learning approach for discovering traffic-flow patterns from speed, traffic volume and occupancy. The implementation converts and validates the raw data, removes 7 invalid rows, standardizes the 3 features and applies K-Means and DBSCAN. On the 17 valid observations, both models recovered the free-flow, moderate and congested states. K-Means achieved Silhouette 0.656 and ARI 0.79 but absorbed both incident-like readings into normal clusters, while DBSCAN achieved Silhouette 0.863 on clustered points, ARI 1.00 and detected both anomalies. These are results on a small demonstration dataset rather than independent test results. Overall, the project provides a reproducible, interactive baseline for traffic-state discovery and shows that density-based clustering is better suited when traffic monitoring must also flag unusual events.

6. Future Work

- Use larger public traffic datasets (for example PeMS or Metro Interstate Traffic Volume) covering multiple roads, days and seasons.
- Add time-of-day, day-of-week, weather and road-type features.
- Select DBSCAN's eps automatically using a k-distance plot with knee-point detection, and report sensitivity to parameter changes.
- Compare additional algorithms such as HDBSCAN, Gaussian Mixture Models, Local Outlier Factor and Isolation Forest.
- Avoid repeated model fitting across tabs in app.py by caching the fitted scaler and models.
- Stream live sensor data and raise alerts automatically when DBSCAN flags noise points.
- Combine clustering with forecasting models to predict congestion before it occurs.

7. References

1. Almatar, K. M. (2023). Traffic congestion patterns in the urban road network: (Dammam metropolitan area). Ain Shams Engineering Journal, 14(3), 101886. doi:10.1016/j.asej.2022.101886.

2. Zang, J., Jiao, P., Liu, S., Zhang, X., Song, G., & Yu, L. (2023). Identifying Traffic Congestion Patterns of Urban Road Network Based on Traffic Performance Index. Sustainability, 15(2), 948. doi:10.3390/su15020948.

3. Košanin, I., Gnjatović, M., Maček, N., & Joksimović, D. (2023). A Clustering-Based Approach to Detecting Critical Traffic Road Segments in Urban Areas. Axioms, 12(6), 509. doi:10.3390/axioms12060509.

4. An, X., Wang, Z., Wang, D., Liu, S., Jin, C., Xu, X., & Cao, J. (2023). STRP-DBSCAN: A Parallel DBSCAN Algorithm Based on Spatial-Temporal Random Partitioning for Clustering Trajectory Data. Applied Sciences, 13(20), 11122. doi:10.3390/app132011122.

5. Liu, H., Yao, R., Cui, C., & Zhao, J. (2024). A Data-Mining Interpretation Method of Pavement Dynamic Response Signal by Combining DBSCAN and Findpeaks Function. Sensors, 24(3), 939. doi:10.3390/s24030939.

6. Berlotti, M., Di Grande, S., & Cavalieri, S. (2024). Proposal of a Machine Learning Approach for Traffic Flow Prediction. Sensors, 24(7), 2348. doi:10.3390/s24072348.

7. Kumari, P. M. K., Manjaiah, D. H., & Ashwini, K. M. (2024). Clustering Algorithms to Analyse Smart City Traffic Data. International Journal of Advanced Computer Science and Applications, 15(8). doi:10.14569/IJACSA.2024.0150811.

8. Zhang, J., & Hu, G. (2024). Transformer model-based multi-scale fine-grained identification and classification of regional traffic states. PeerJ Computer Science, 10, e2625. doi:10.7717/peerj-cs.2625.

9. Lahijanian, Z., Isaenko, N., Fusco, G., & Colombaroni, C. (2025). Empirical analysis of traffic patterns: Leveraging machine learning techniques for in-depth insights. Transportation Research Procedia. https://www.sciencedirect.com/science/article/pii/S2352146525005757.

10. Khasawneh, M., & Awasthi, A. (2026). Clustering Based Approach for Enhanced Characterization of Anomalies in Traffic Flows. Future Transportation, 6(1), 11. doi:10.3390/futuretransp6010011.

11. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96), 226–231.

12. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Signature of the Guide **Course Instructor**